* Context free Grammars *

(N, T, P, S) → Generators of languages

(V, Σ, P, S) → Automata → identifiers or recogn

N|V → Set of Non-terminals

T|Σ → set of terminals

P → Set of Productions

S → starting Non-terminal

A production is of type $\alpha \rightarrow \beta$

α → single Non terminal

β → combination of terminal & non-terminal

$\alpha \in N$

$\beta \in (N \cup T)^*$

$N = \{S\}$, $\Sigma = \{a, b\}$

Ex-1 :- P : $S \rightarrow aSa$ | bSb | ϵ — even length palindromes

Ex-2 :- $S \rightarrow AB$ → $a^m b^n$

$A \rightarrow aA / \epsilon$

$B \rightarrow bB / \epsilon$

$N = \{S, A, B\}$

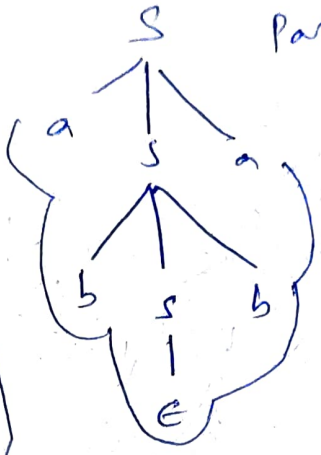
$T = \{a, b\}$

Ex-1 :-

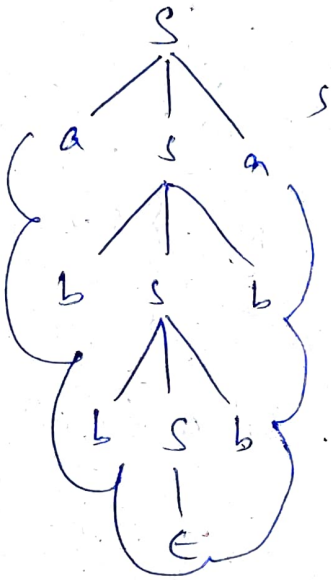
Leftmost Derivation

$S \rightarrow aSa$
$\rightarrow abSba$
$\rightarrow abEba$
$\rightarrow abba$

Parse tree



abba



$S \rightarrow aSa$
$\rightarrow abSba$
$\rightarrow abbSbba$
$\rightarrow abbEbba$
$\rightarrow abbbba$

abbbba

2 types of Derivations

→ Left most derivation (LMD)
→ Right most derivation (RMD)

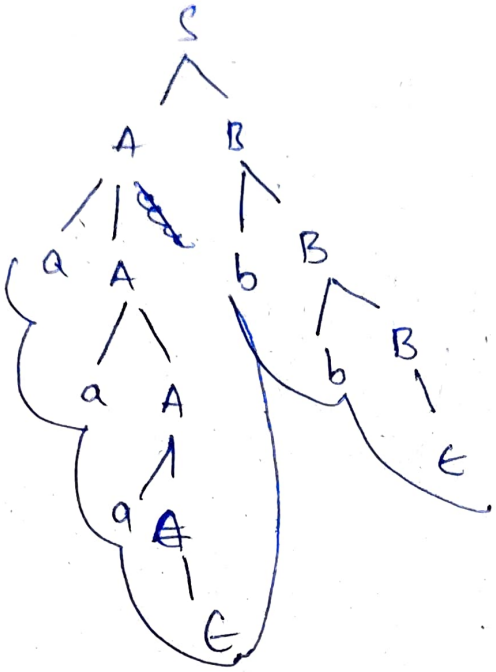
Ex-2 :- aaabb

LMD

$S \rightarrow AB$
$\rightarrow aAB$
$\rightarrow aaAB$
$\rightarrow aaaAB$
$\rightarrow aaaEB$
$\rightarrow aaaB$
$\rightarrow aaabB$
$\rightarrow aaabbB$
$\rightarrow aaabbE$
$\rightarrow aaabb$

RMD

$S \rightarrow AB$
$\rightarrow AbB$
$\rightarrow AbbB$
$\rightarrow AbbE$
$\rightarrow aAbb$
$\rightarrow aaAbb$
$\rightarrow aaaAbb$
$\rightarrow aaaEbb$
$\rightarrow aaabb$



Ex-2:-

Q

$S \rightarrow 0S \mid 1AA$

$A \rightarrow 0 \mid 1A \mid 0B$

$B \rightarrow 1 \mid 0BB$

$N = \{S, A, B\}$

$T = \{0, 1\}$

S: 0 1 0 0 1 1 0

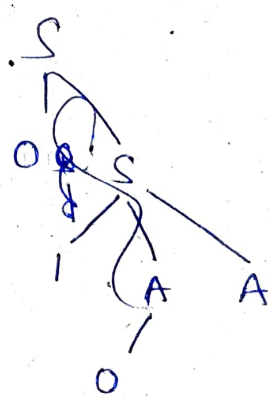
A B

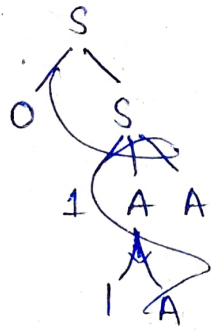
$S \rightarrow 0S$

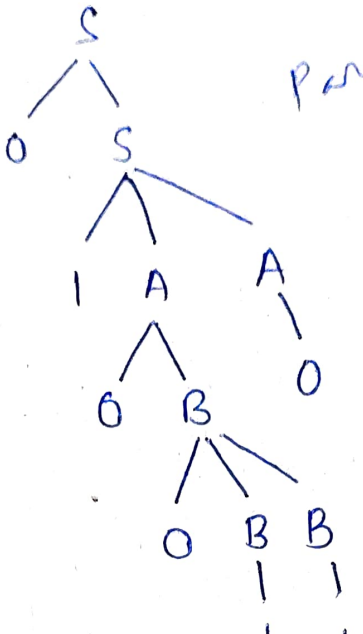
$\rightarrow 01AA$

$\rightarrow 01$









LMD :-

$S \rightarrow 0S$
$\rightarrow 01AA$
$\rightarrow 010BA$
$\rightarrow 0100BBA$
$\rightarrow 0100111A$
$\rightarrow 0100110$

RMD :-

$S \rightarrow 0S$
$\rightarrow 01AA$
$\rightarrow 01A0$
$\rightarrow 010B0$
$\rightarrow 0100BB0$
$\rightarrow 0100110$

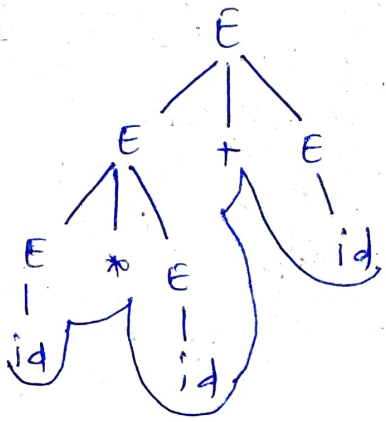
Q Ex.

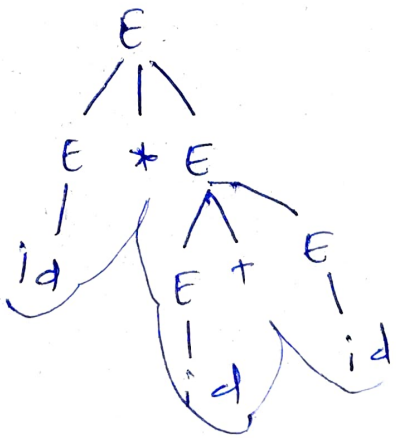
$E \rightarrow E+E$
$\mid E*E$
$\mid id$

$N = \{E\}$

$T = \{id, +, *\}$

string :- $id * id + id$





LMD:-

$E \rightarrow E + E$

$\rightarrow E * E + E$

~~$\rightarrow id * id + id$~~

$\rightarrow id * E + E$

$\rightarrow id * id + E$

$\rightarrow id * id + id$

RMD:-

$E \rightarrow E + E$

$\rightarrow E + id$

$\rightarrow E * E + id$

$\rightarrow E * id + id$

$\rightarrow id * id + id$

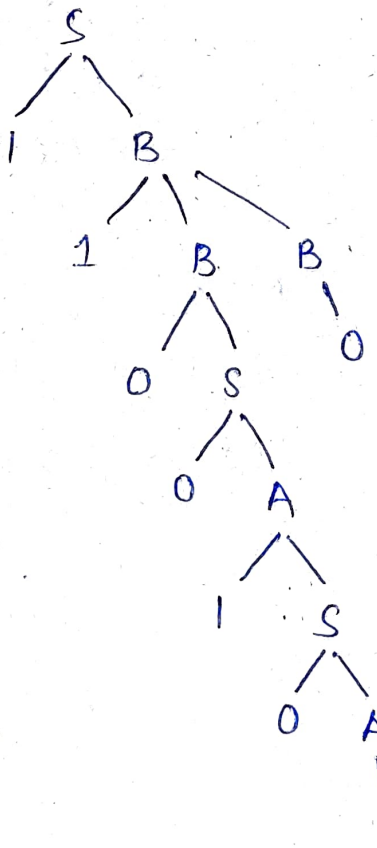
Q :-

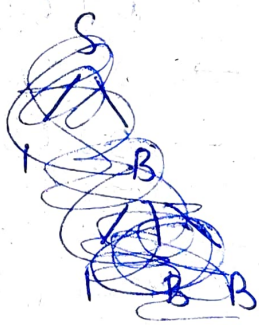
$S \rightarrow 1B \mid 0A$

$A \rightarrow 1 \mid 1S \mid 0AA$

$B \rightarrow 0 \mid 0S \mid 1BB$

string :- 11001010





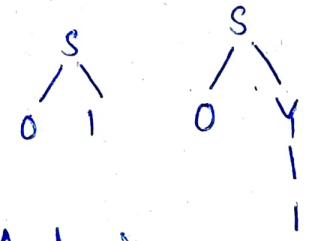
1Q)

$S \rightarrow 0Y \mid 01$

$X \rightarrow 0XY \mid 0$

$Y \rightarrow XY1 \mid 1$

string :- 01



Ambegious.

2Q)

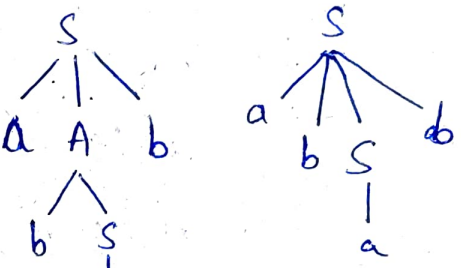
$S \rightarrow a \mid abSb \mid aAb$

$A \rightarrow bS \mid aAAb$

string :- a

S — a

string :- abab



Ambigous

$L_1 = \{ a^m b^n \mid m, n \geq 0 \}$

$S \rightarrow AB$

$A \rightarrow aA \mid \varepsilon$

$B \rightarrow bB \mid \varepsilon$

$L = \{ \varepsilon, a, b, aa, ab, ba, bb, \ldots \}$

$L_2 = \{ a^m b^n \mid m, n > 0 \}$

$S \rightarrow AB$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid b$

$L_3 = \{ a^m b^n \mid m > 0, n \geq 0 \}$

$S \rightarrow AB$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid \varepsilon$

$L_4 = \{ a^m b^n \mid m, n > 1 \}$

$S \to AB$

$A \to aA \mid aa$

$B \to bB \mid bb$

$L_5 = \{ a^m b^n c^p \mid m, n, p \geq 0 \}$

$S \to ABC$

$A \to aA \mid \varepsilon$

$B \to bB \mid \varepsilon$

$C \to cC \mid \varepsilon$

$L_6 = \{ a^n b^n \mid n \geq 0 \}$

$S \to A$

$A \to aSb \mid \varepsilon$

$\approx \quad S \to aSb \mid \varepsilon$

$L_7 = \{ (ab)^n \mid n \geq 0 \}$

$S \to abS \mid \varepsilon$

$L_8 = \{ a^n b^n \mid n > 0 \}$

$S \to aSb \mid ab$

$L_9 = \{ a^m b^n a^m \mid m, n \geq 0 \}$

$S \to aAa \mid \varepsilon \mid b$

$A \to aAa \mid \varepsilon \mid B$

$B \to bB \mid \varepsilon$

$S \to ASB$

$A \to aA \mid a$

$B \to bB \mid a \mid b \mid \varepsilon$

$L_{10} = \{a^m b^m c^n \mid m, n \geq 0\}$

$S \to aXb \mid \varepsilon \mid C$

$X \to aXb \mid \varepsilon \mid C$

$C \to cC \mid \varepsilon$

$S \to AC \mid \varepsilon$

$A \to aAb \mid \varepsilon$

$C \to cC \mid \varepsilon$

$L_{11} = \{a^m b^n c^n \mid m, n \geq 0\}$

$S \to AB \mid \varepsilon$

$A \to aA \mid \varepsilon$

$B \to bBc \mid \varepsilon$

$L_{12} = \{a^m b^m c^n d^n \mid m, n \geq 0\}$

$S \to AB \mid \varepsilon$

$A \to aAb \mid \varepsilon$

$B \to cBd \mid \varepsilon$

$L_{13} = \{a^m b^n c^n d^m \mid m, n \geq 0\}$

$S \to AB \mid \varepsilon$

$A \to aAd \mid aBd \mid \varepsilon$

$B \to bBc \mid \varepsilon$

$L_{14} = \{\omega \# \omega^R \mid \omega \in (a+b)^*\}$

$S \to ACB$

$A \to aSa$

$C \to \#$

$B \to bSb$

$S \to aSa \mid bSb \mid \#$

$L = \{0^m 1^n 0^{m+n}, m,n \geq 1\}$

$\approx L = \{0^m 1^n 0^n 0^m\}$

$S \rightarrow 0S0 / 0N0$

$N \rightarrow 1N0 / 10$

$L_{15} = \{a^n b^n c^n / n \geq 0\}$ — Context Sensitive Languages

$L_{16} = \{a^m b^n c^m d^n / m,n \geq 0\}$ — Not CFL

$L_{17} = \{w \# w / w \in (a+b)^*\}$

** Closure Properties of CFL

① CFL's are closed under union, concatenation & kleene Closure

② CFL's are not closed under intersection, complement

→ If L_1, L_2 are CFL's

G_1, G_2 are CFG's

$G_1 (N_1, T, P_1, S_1)$

$G_2 (N_2, T, P_2, S_2)$

i) $L_1 \cup L_2 = \{x / x \in L_1 \text{ or } x \in L_2\}$

$P_1 : S_1 \rightarrow \ldots$ $P_2 : S_2 \rightarrow \ldots$

$S \rightarrow S_1 / S_2$

$G : (\{N_1 \cup N_2 \cup S\}, T, P_1 \cup P_2 \cup S \rightarrow S_1 | S_2, S)$

So, union of any two CFLs is always a CFL

i.e, CFLs are closed under union

ii) $L_1 L_2 = \{xy \mid x \in L_1 \wedge y \in L_2\}$

$G : (\{N_1 \cup N_2 \cup S\}, T, P_1 \cup P_2 \cup S \rightarrow S_1 S_2, S)$

So, concatenation of two CFLs is always a CFL

i.e, CFLs are closed under concatenation

If L is CFL & G is CFG

$G (N, T, P, S)$

iii) $L^* :- \quad S' \rightarrow SS' | \epsilon$

$G : (\{N \cup S'\}, T, P \cup S' \rightarrow SS' | \epsilon, S')$

CFLs are closed under Kleene Closure

$L_1 = \{a^m b^m c^n \mid m, n \geq 0\}$ a's & b's are Equal

$L_2 = \{a^m b^n c^n \mid m, n \geq 0\}$ b's & c's are Equal

$L_1 \cap L_2 = \{a^k b^k c^k \mid k \geq 0\}$

↳ a's, b's, c's R Equal

∴ Although L_1 & L_2 are CFL's here,

$L_1 \cap L_2$ is not a CFL

* Intersection of any two CFL is not always a CFL

* CFLs are not closed under Intersection

↳ may / may not be

De Morgan's Law:-

$$(L_1 \cup L_2)^C = L_1^C \cap L_2^C$$

Assume CFLs are closed under complement.

If L_1 & L_2 are CFL, $L_1 \cup L_2$ is also a CFL as per closure property on union

As per assumption, $(L_1 \cup L_2)^C$ is a CFL,

L_1^C, L_2^C are also CFL's.

If $(L_1 \cup L_2)^C$ is CFL then,

$L_1^C \cap L_2^C$ is also CFL

⇒ But,

intersection of 2 CFL's is a CFL which leads to contradiction.

our assumption is wrong.

CFL's are not closed under compliment

$L = \{ a^m b^n c^p \mid m=n \text{ (or) } n=p \}$

Write Grammar?

$L = L_1 \cup L_2 \Rightarrow$ $L_1 = \{ a^m b^n c^p \mid m=n \}$

$L_2 = \{ a^m b^n c^p \mid n=p \}$

$L_1 \Rightarrow$ ~~$S \to \epsilon \mid aS_1b$~~

$S_1 \to AB \mid \epsilon$

$A \to aAb \mid ab$

$B \to cB \mid c$

$L_2 \Rightarrow S_2 \to CB \mid \epsilon$

$C \to aC \mid a$

$D \to bDc \mid bc$

Grammar:- $L_1 \cup L_2 = L$

$L(L_1 \cup L_2) \Rightarrow$

$L \Rightarrow S \to S1 \mid S2$

$S1 \to AB \mid \epsilon$

$A \to aAb \mid ab$

$B \to cB \mid c$

$S_2 \to CD \mid \epsilon$

$C \to aC \mid a$

$D \to bDc \mid bc$

$S \to aSb$

$S \to S_1 \mid S_2$

* Push Down Automata (PDA)

$M = (Q, \Sigma, T, \delta, q_0, z_0, F)$

Q: Set of States

Σ: Set of ~~Alphabets~~ Input Alphabet

T: Stack alphabet

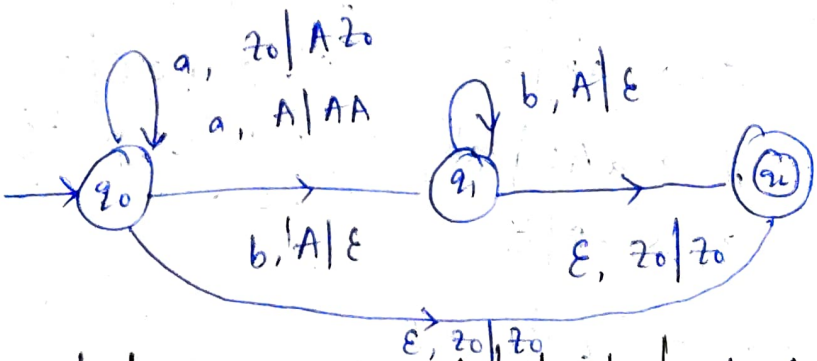
δ: Transition function

q_0: Initial state

z_0: Initial stack symbol

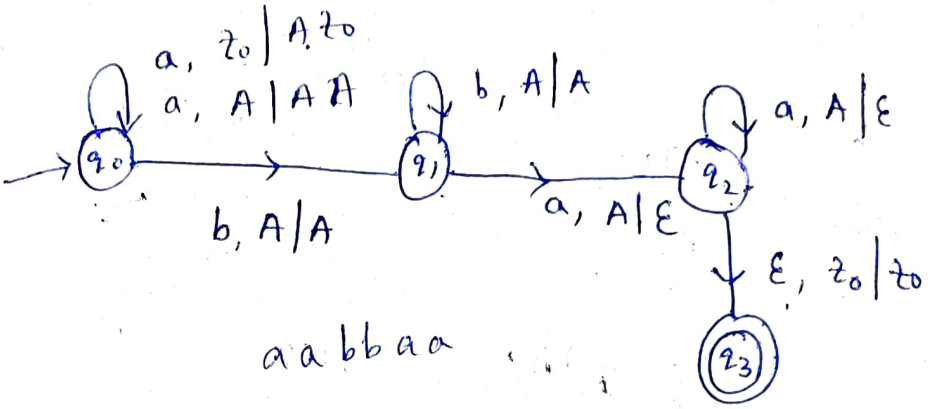
F: Final state

$L_1 = \{a^n b^n \mid n \geq 0\}$ ε



present state	i/p Symbol	stack top	Next state	stack top
q_0	a	z_0	q_0	$A z_0$
q_0	a	A	q_0	AA
q_0	ε	z_0	q_2	z_0
q_0	b	A	q_1	ε
q_1	b	A	q_1	ε
q_1	ε	z_0	q_2	z_0

$L_2 = \{ a^m b^n a^m \mid m, n > 0 \}$



aabbaa

$L_3 = \{ w \# w^R \mid w \in (a+b)^* \}$

$L_4 = \{ w \mid w$ has equal no. of a's & b's $\}$

$L_5 = \{ w \mid w$ has balanced parantheses $\}$

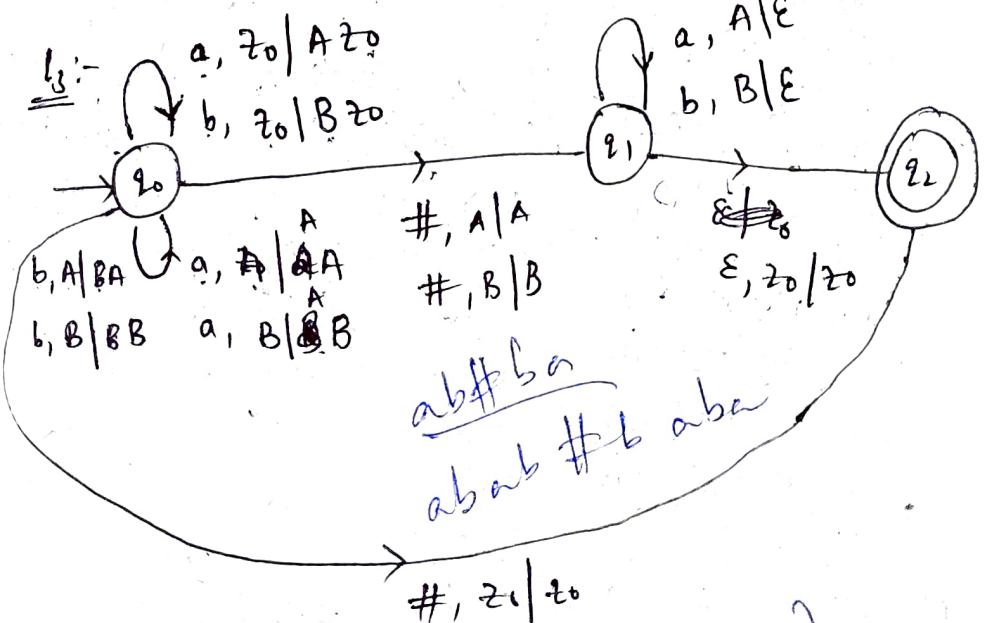
↳ (( )) ✓

((( )) ✓

(( ) ( ) ✓

) (✗

L_3 :-

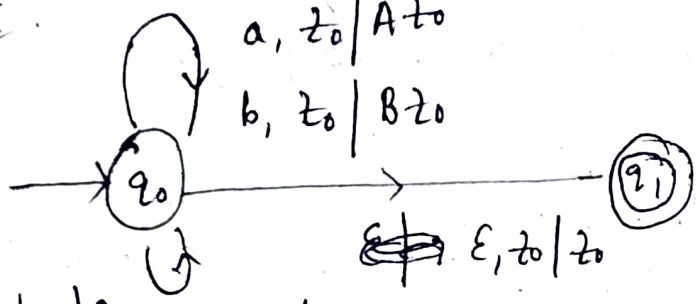


ab#ba

abab#baba

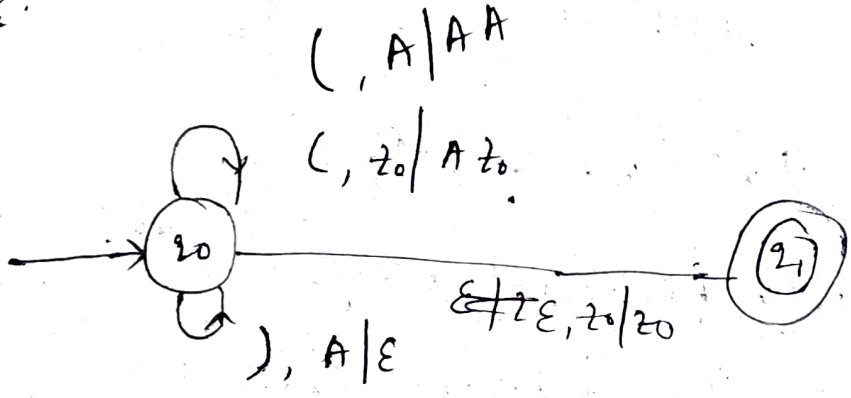
$\delta(q_0, \#, z_0) = (q_1, z_0)$

L4

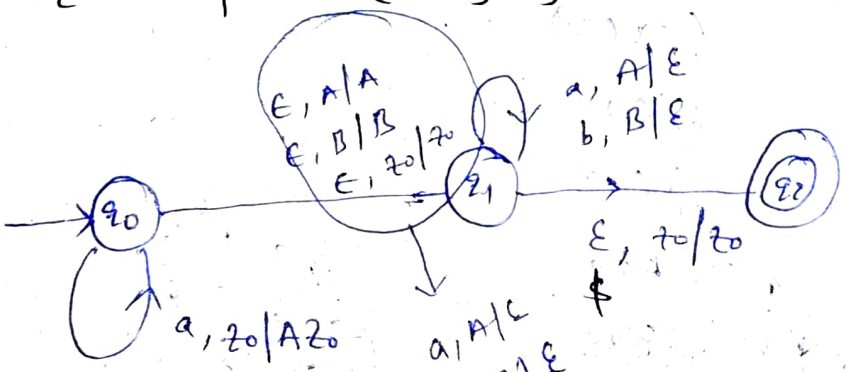


b, A | ε
b, B | bB
a, A | aA
a, B | ε

L5 :-



L6 :- $\{ w w^R \mid w \in (a+b)^* \}$:- NPDA



abba $

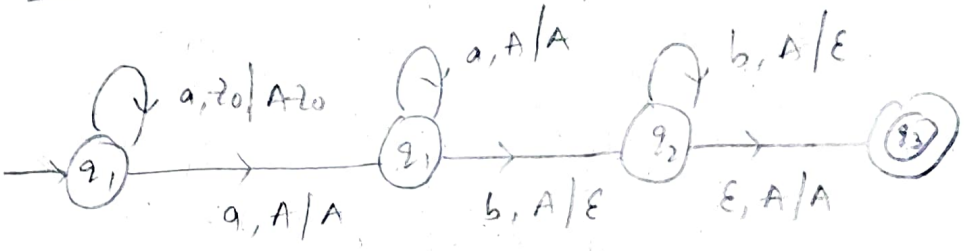
NPDA. Expressive power is not same as DPDA

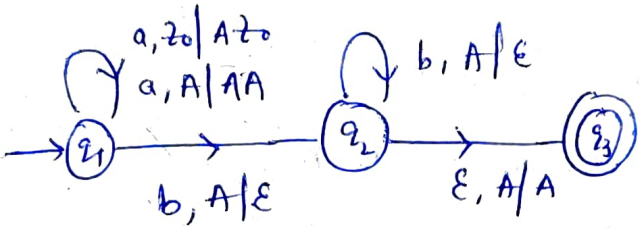
NPDA ≠ DPDA

$L_7 :- \{a^m b^n \mid m > n, m, n > 0\}$

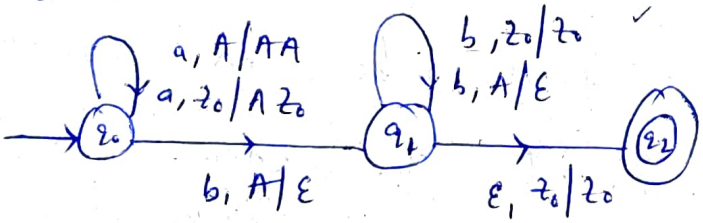
$L_8 :- \{a^m b^n \mid m \leq n, m, n > 0\}$

Language 7 :-



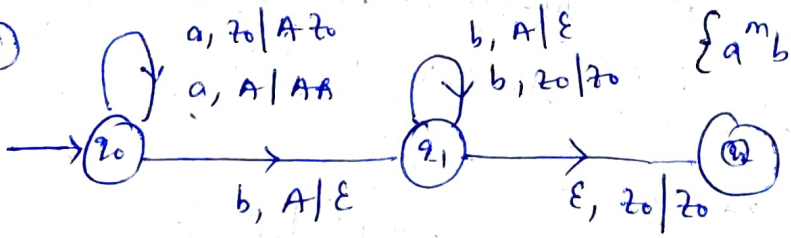


Language – 8 :-



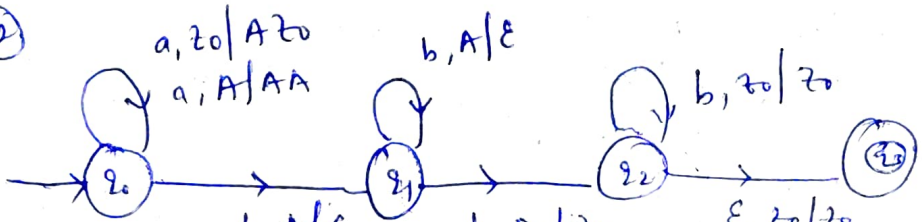
---

(1)



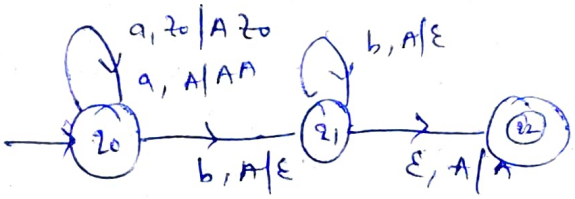
$\{a^m b^n \mid m \leq n, m, n > 0\}$

(2)



$\{a^m b^n \mid m < n\}$

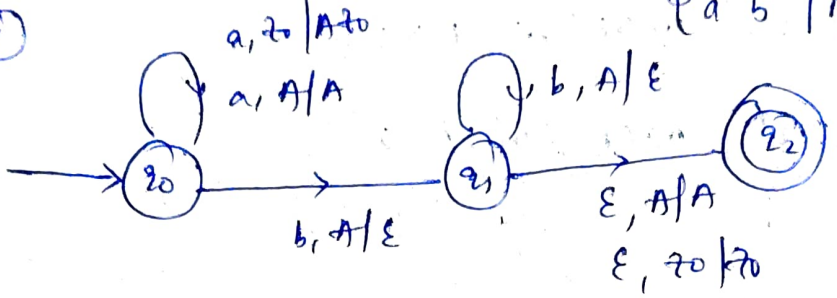
(3)



$\{a^n b^n \mid m > n\}$

ganesh

(a) $\{a^m b^n \mid m \geq n\}$



Normal forms of CFG's.

There are 2 standard forms to express CFG.

① Chomsky Normal form.

② Greibach Normal form

* A CFG is said to be in Chomsky NF if all the productions are of the form $N \to N_1 N_2$ (string of exactly 2 NT)

$N \to T$ (single terminal).

Ex: $S \to AB$

$A \to A_1 A \mid a$

$A_1 \to a$

$B \to B_1 B \mid b$

$B_1 \to b$

$\{a^m b^n \mid m, n > 0\}$

$A \to a$ (unit prod)

$S \to aSa \mid a$

$S \to aSa$

$A \to a$

$S \to ASA$

$S \to AA'$

$A' \to SA$

$S \to AA' \mid a$

$A' \to SA$

$A \to a$

* A grammar is said to be GNF if all the productions are of the form $N \to$ (Single Terminal)(string of NT's)

Eg:- $N \to T$.

$S \to aB$

$B \to bB \mid b$

} each time a string is generated

* Pumping Lemma for CFL

Let L be a CFL, then we can find a natural number n (pumping lemma const) such that every z in L whose length is atleast l, can be written as

$z = uvwxy$ satisfying

① $|vx| \geq 1$

② $|vwx| \leq n$

③ $\forall_{i \geq 0}$ $uv^iwx^iy \in L$

* Chomsky Hierarchy of Grammar

Regular Grammar :- (Type-3)

A grammar is said to be RG if all the productions are of the form :

$A \to \alpha$ (or) $A \to \alpha B$ where $\alpha \in T$; $A, B \in$

$A \to aA \mid \varepsilon$ $\quad S \to aS \mid \varepsilon$

*

Type-2 CFG :-

A grammar is CFG if the productions are of the form $A \to \alpha$, where $A \in N$

$\alpha \in (NUT)^*$

** Type-1 CSG :-

A grammar is CSG if productions are

$\alpha A \beta \longrightarrow \alpha \gamma \beta$; $A \in N$ $\quad \alpha, \gamma, \beta \in (NUT)^*$

Eg :- $aA \longrightarrow aAb$ $\quad bcS \to bSc \mid aA$

$aA \to aAb \mid \varepsilon$

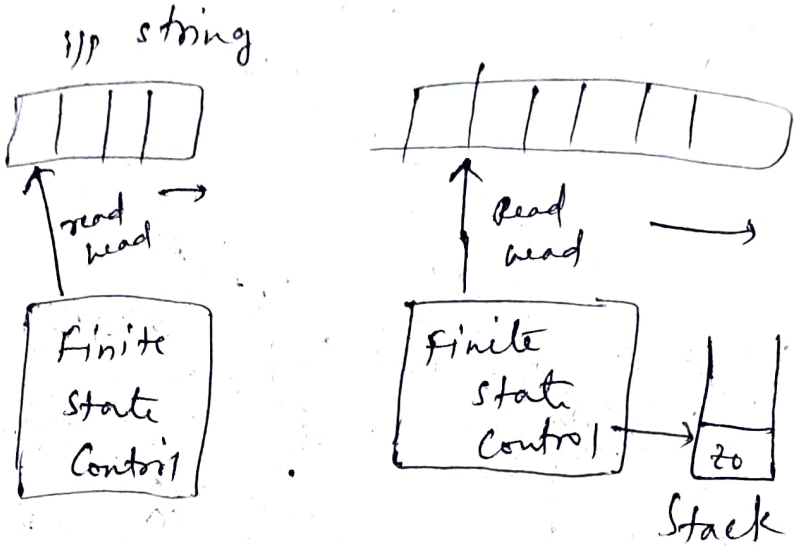
** Type-0 Unrestricted Grammar

productions are of the form :-

$(L_c)(N)(R_c) \to$ String of $T / NT /$ Both

$L_c \to$ Left content

$R_c \to$ Right content



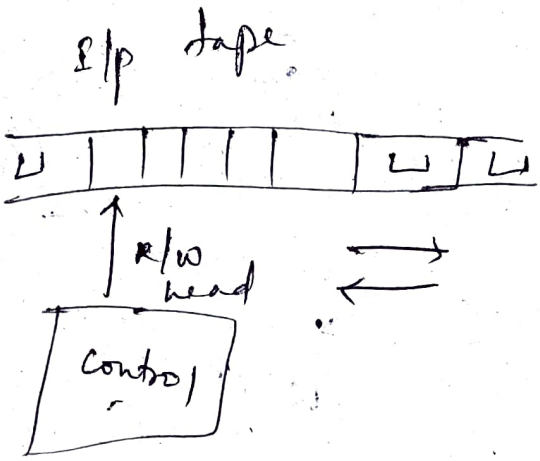
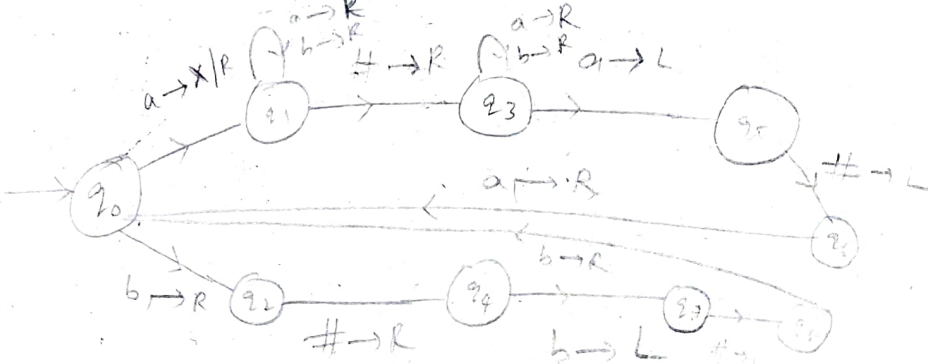


abb # abb

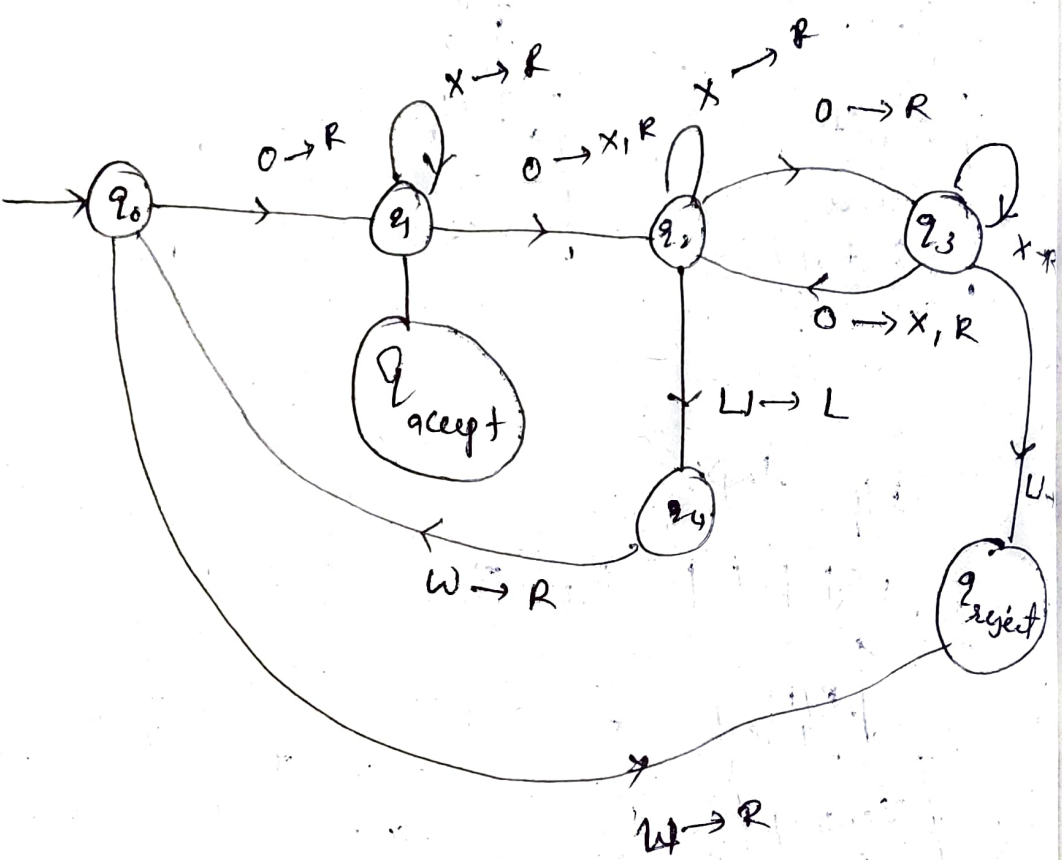
$L = \{ w\#w \mid w \in (a+b)^* \}$

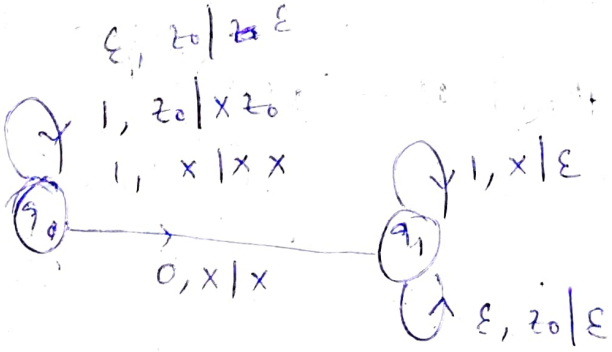


aa#aab

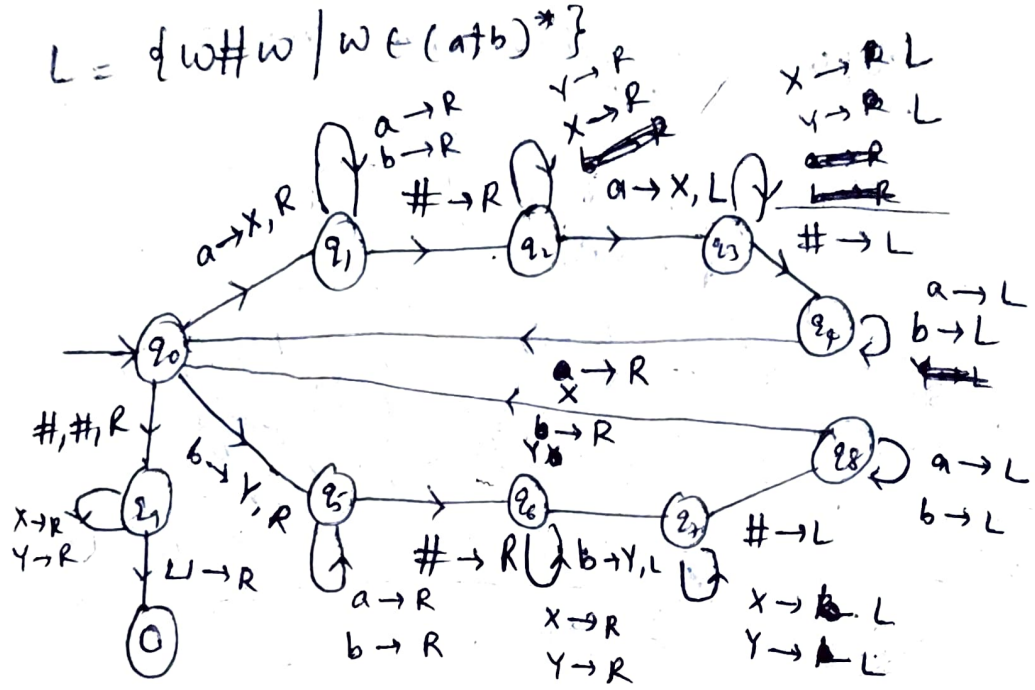
$L_2 = \{0^{2^n} \mid n \geq 0\}$

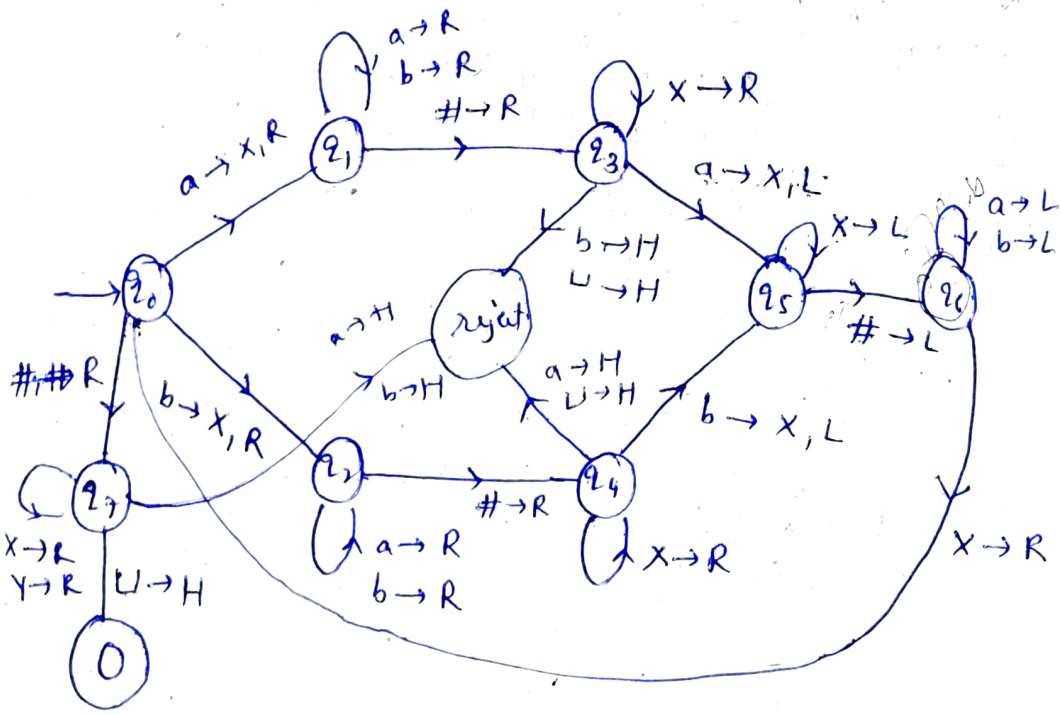
0 0 0 0
0 X 0 X



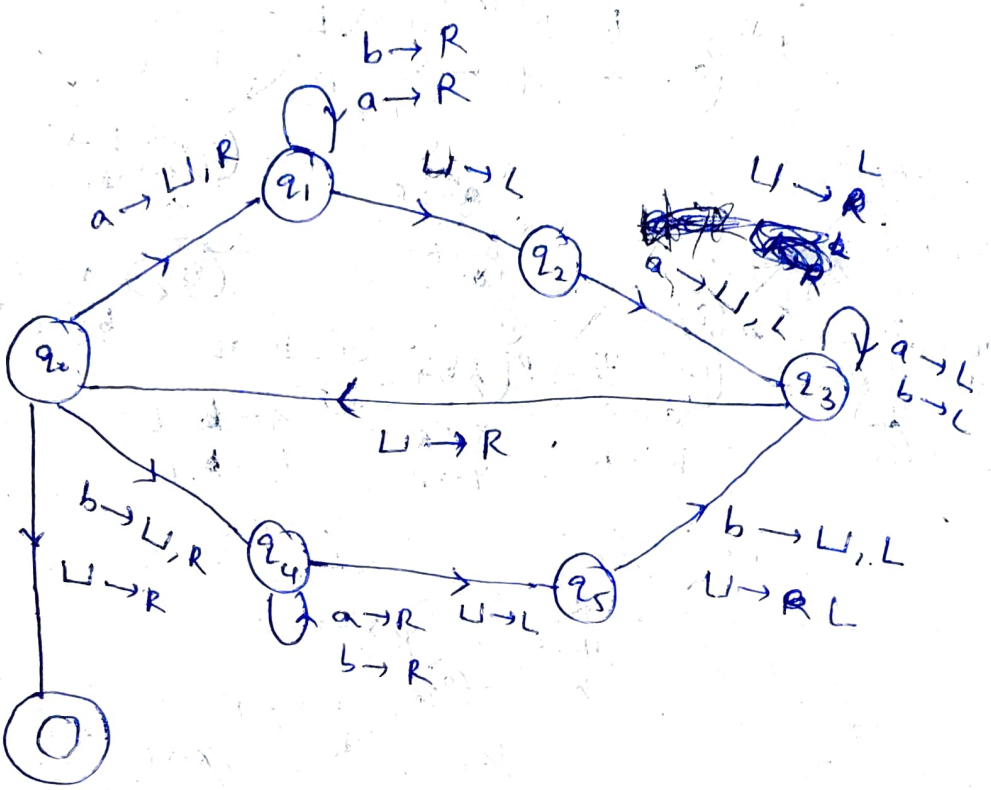


$L = \{ w\#w \mid w \in (a+b)^* \}$



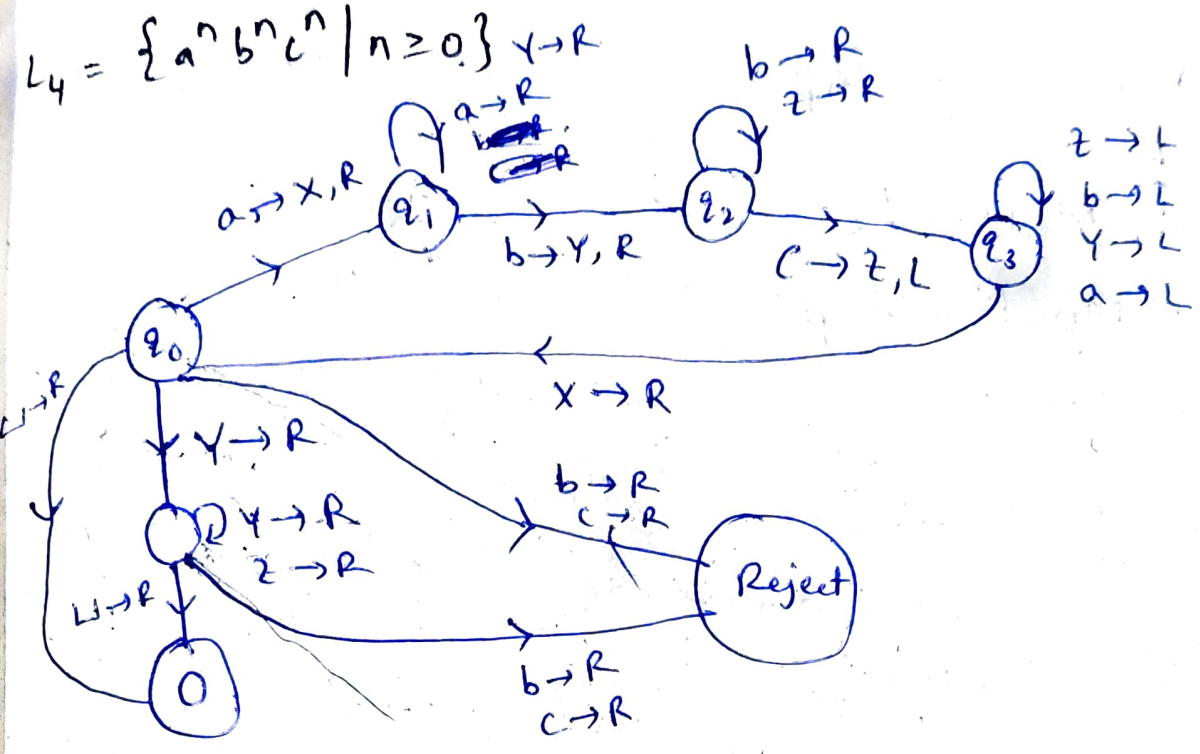


$L_3 = \{ w | w$ is a palindrome over $\{a, b\} \}$





$L_4 = \{a^n b^n c^n \mid n \geq 0\}$



* Variation/Types of Turing Machines

1. Multi-head TM
2. Multi-tape TM
3. Two-way infinite tape TM
4. k-dimensional TM
5. Non-deterministic TM
6. Enumerator

time & Efficiency

Determin. — poly

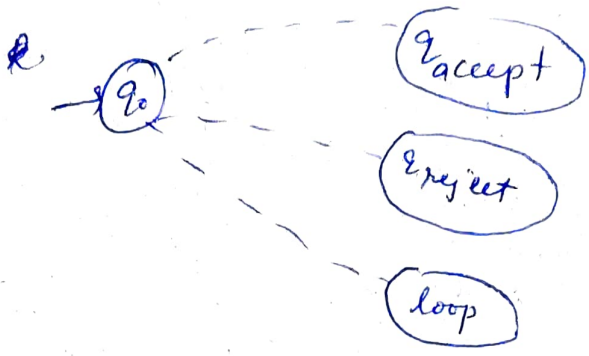
No-Deterministic — Non poly

No — Turing — Undecidable

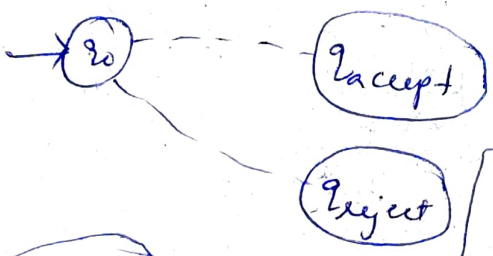
* Church Turing Thesis

1. Turing Decidable Languages :-
   (or) Recursive languages.
2. Turing Recognizable language :-
   (or) Recursive Enumerable lang

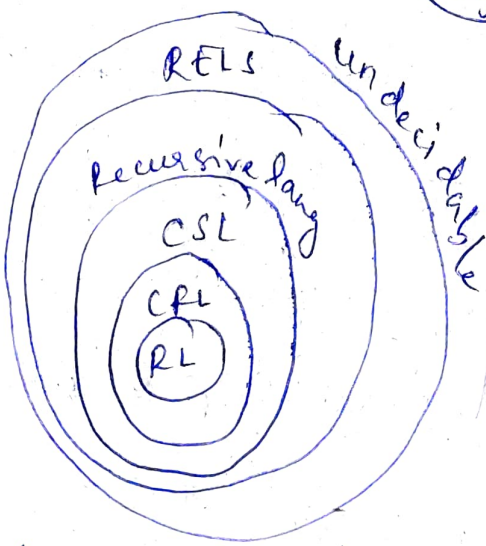
REL's / Turing Acceptable / Turing Recognizable



Recursive / Turing Decidable

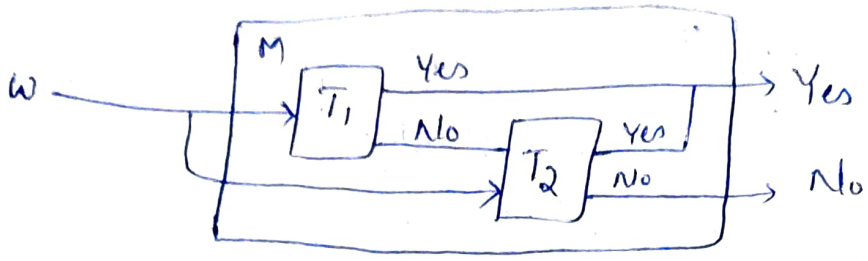


Halting is Undecidable

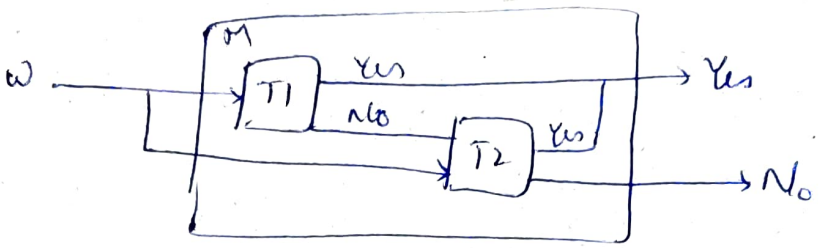


* Closure properties of Recursive & RELs.

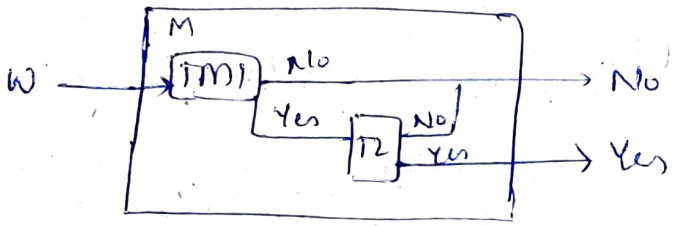
① union of 2 Recursive languages is Recursive.



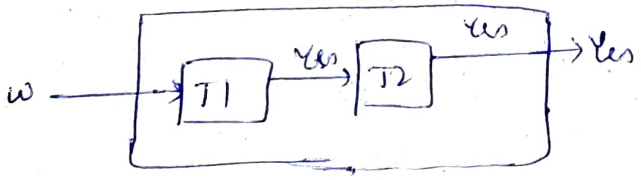
(2) Union of 2 REL's is REL.



(3) Intersection of 2 Recursive is Recursive



(4) Intersection of 2 REL's is REL.



(5) Complement of Recursive lang is Recursive



(6) If a language & its Complement both are recursively Enumerable then both the language & its complent are recursive

Yes because As RELs give only where the Machine halts but don't give where

So, complement of REL is not always REL but its subset complement of Recursive is Recursive.

So, if lang & its complement are RELs means they are Recursive

(7) Concatenation of 2 RELs is REL

If L_1 & L_2 are RELs

Eg:- $L_1 = \{a^n\}$ $L_2 = \{b^n\}$.

$L = \{a^n b^n\}$. — REL

(8) Concatenation of 2 Recur is Recursive

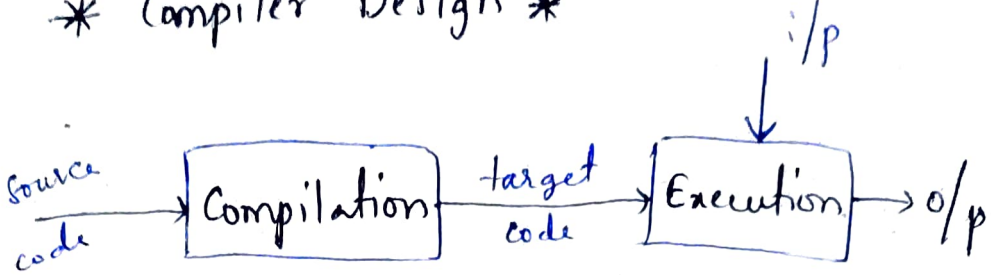
(9) Kleene star on REL is REL

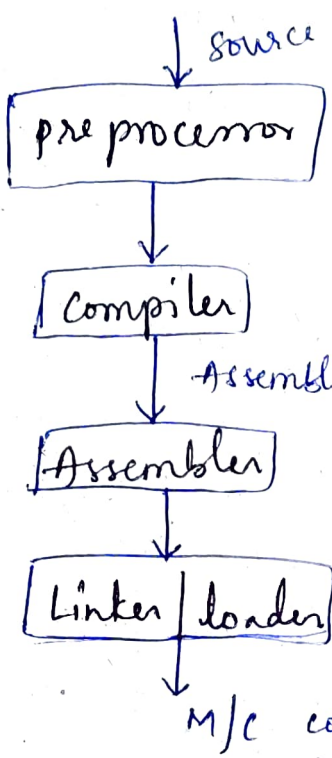
(10) Kleene star on Recur is Recursive

(11) Intersection of Recursive & REL is REL.

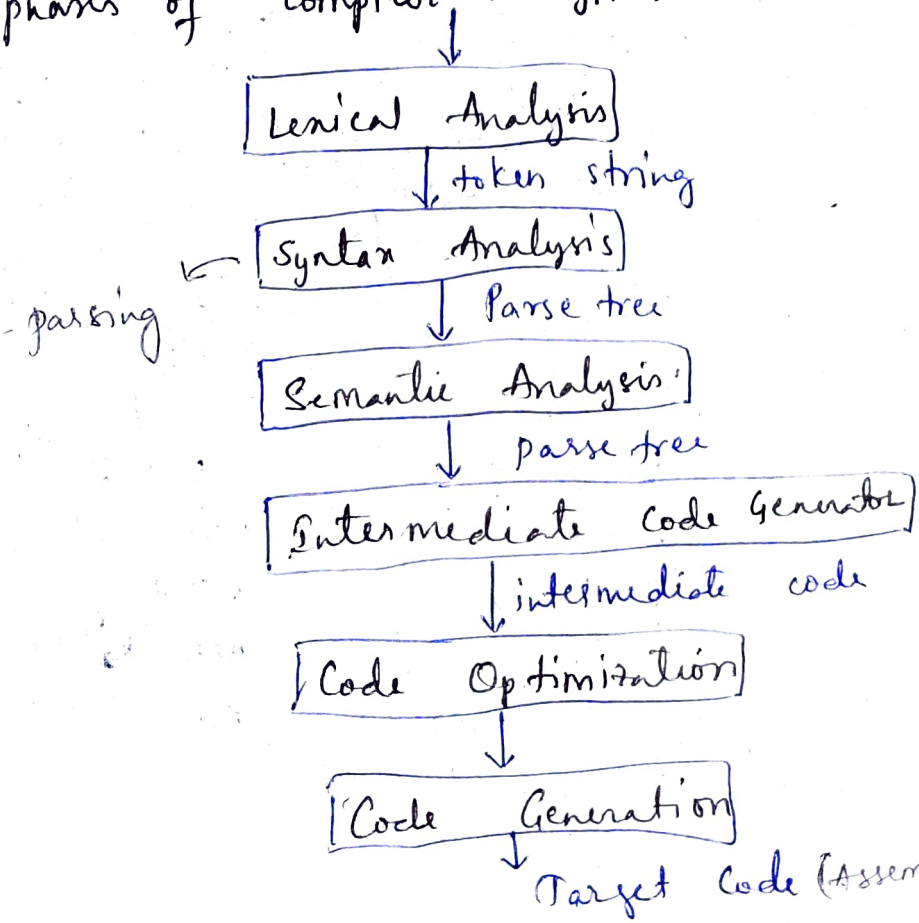
(12) If L is Recursive, then $\Sigma^* - L$ is also Recursive

* Compiler Design *





phases of Compiler Design.



float a, b, c;

c = a + b * 5;



↓

LA ← generates

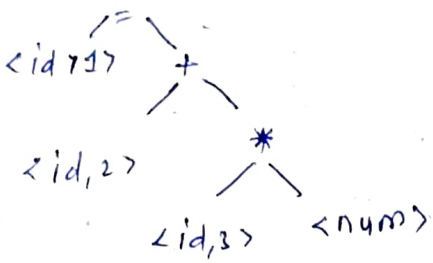
↓

<id,1> = <id,2> + <id,3> * <num>

↓

Syntax A

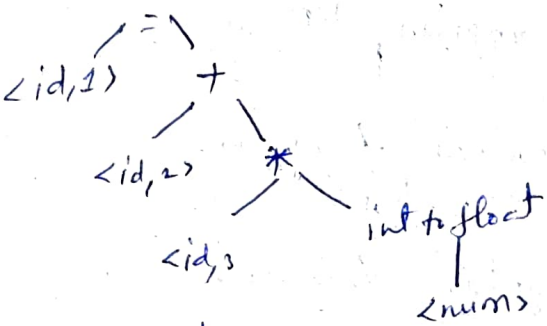
↓



↓

Semantic A

↓



↓

ICG

t_1 = inttofloat (5)

t_2 = b * t_1

t_3 = a + t_2

c = t_3

↓

Code Op

t_2 = b * 5.0

c = a + t_2

→ Code Generator

↓

LD R_1, b

MUL R_1, 5.0

LD R_2, a

ADD R_2, R_1

STR R_1, c

Errors :-

1. Missing Semicolon
2. Type mismatch
3. Assignment instead of comparision
4. Format Specifier mismatch
5. undeclared variable.
6. Multiple times declaring same variable
7. Break, continue outside loops, switch statements
8. Wrong no. of function arguments.
9. Using variable/functions without declaration
10. Buffer overflow
11. Wrong return type
12. Modifying const variables.
13. Not use static in functions parameters.

* Lexical Analysis

The main task of the lexical analyzer is to read the i/p characters of the source program group them into lexims & produce as o/p a sequence of tokens for each lexim in the source program. The stream of tokens is sent to the parser for syntax analysis

* Token Categories

Lexims can be categorized into following:

(1) One token for each keyword.

if – token int – token

else – token

(2) Identifier – variable & function names

(3) Operators

Arithmetic (+, /, *, %, -)

Relational (>, <, =, >=, <=, ≠)

logical (!, &&, ||)

Bitwise

Assignment (=)

(4) Numbers — 1, -1, 1.23, 0.5, -0.0023

1E3

(5) Special Symbols ({, }, ; .
apart from operators & above are special

(6) literals — "a" → is literal
↳ No Syntax Check.

```
void main()
{
    int a,b,c;
    a=10;
    printf("a=%d",a);
}
```

keyword — Sequence of Symbols is the pattern

Identifiers :—

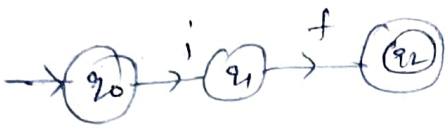
letter_ (letter_ | digit)*

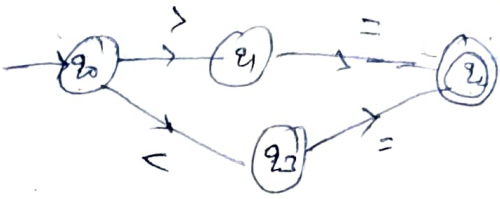
Regular Definations

letter_ → A|B|C|...|Z|a|b...|z|_

digit → 0|1|2|...|9

identifier letter_ (letter_ | digit)*





Numbers :-

sign = +/-/ε

digit = 0/1/2/3 ... /9 ⇒

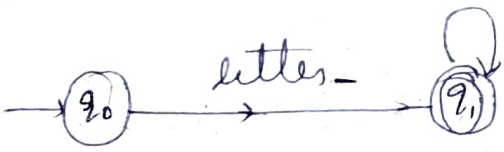
~~fractions~~ :

digits → digit (digit)* 2e3 ⇒ 2x10^3

optional fraction → . digit / ε

optional Exponent → E (+/-/ε) (digits) / ε

numbers ⇒ sign digits optinal fraction . optional Expon



panic mode error recovery

if LA can't proceed further

existly seq of number not matching any defined pattern.

Remove 1 symbol

add 1 symbol

transpose symbols.